

Problem 1. Consider the following string of ASCII characters that were captured by Wireshark when the browser sent an **HTTP GET** message (i.e., this is the actual content of an **HTTP GET** message). The characters `<cr>` and `<lf>` are carriage return and line-feed characters (that is, the italicized character string `<cr>` in the text below represents the single carriage-return character that was contained at that point in the HTTP header). Answer the following questions, indicating where in the **HTTP GET** message below you find the answer.

```
GET /cs453/index.html HTTP/1.1<cr><lf>Host: gaia.cs.umass.edu<cr><lf>
User-Agent: Mozilla/5.0 (Windows;U; Windows NT 5.1; en-US; rv:1.7.2)
Gecko/20040804 Netscape/7.2 (ax)<cr><lf> Accept: ext/xml,
application/xml, application/xhtml+xml, text/html;q=0.9,
text/plain;q=0.8, image/png, */*;q=0.5
<cr><lf>Accept-Language: en-us,en;q=0.5<cr><lf>Accept-Encoding:
zip, deflate<cr><lf>Accept-Charset: ISO-8859-1, utf8;q=0.7,*;q=0.7<cr><lf>Keep-Alive: 300<cr><lf>
Connection: keep-alive<cr><lf><cr><lf>
```

- What is the URL of the document requested by the browser?
- What version of HTTP is the browser running?
- Does the browser request a non-persistent or a persistent connection?
- What is the IP address of the host on which the browser is running?
- What type of browser initiates this message? Why is the browser type needed in an HTTP request message?

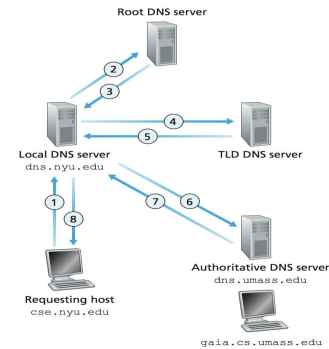
Problem 2. Assume that the RTT between a client and the local DNS server is RTT_l , while the RTT between the local DNS server and other DNS servers is RTT_r . Assume that no DNS server performs caching.

- What is the total response time for the scenario illustrated in the following Figure?

- A) RTT_l
 B) $RTT_l + RTT_r$
 C) $RTT_l + 2RTT_r$
 D) $RTT_l + 3RTT_r$

- Assume now that the DNS record for the requested name is cached at the local DNS server. What is the total response time?

- A) RTT_l
 B) $RTT_l + RTT_r$
 C) $RTT_l + 2RTT_r$
 D) $RTT_l + 3RTT_r$



Problem 3. Suppose within your browser you type in a URL to obtain a webpage. The IP address for the associated URL is not cached in your local host, so as DNS lookup is necessary to obtain the IP address. Suppose that n DNS servers are visited before your host receives the IP address from DNS. Visiting the i -th of them incurs an RTT_i per DNS. Further suppose the HTML file references eight very small objects on the same server. Let RTT_o denote the RTT between the local host and the server containing an object. Neglecting transmission times, how much time elapses with

- Non-persistent HTTP with no parallel TCP connections?

- A) $RTT_1 + \dots + RTT_n + 8RTT_o$
 B) $RTT_1 + \dots + RTT_n + 9RTT_o$
 C) $RTT_1 + \dots + RTT_n + 16RTT_o$
 D) $RTT_1 + \dots + RTT_n + 18RTT_o$

- Non-persistent HTTP with the browser configured for 5 parallel connections?

- A) $RTT_1 + \dots + RTT_n + 4RTT_o$
 B) $RTT_1 + \dots + RTT_n + 5RTT_o$
 C) $RTT_1 + \dots + RTT_n + 6RTT_o$
 D) $RTT_1 + \dots + RTT_n + 7RTT_o$

- Persistent HTTP?

- A) $RTT_1 + \dots + RTT_n + 8RTT_o$
 B) $RTT_1 + \dots + RTT_n + 10RTT_o$
 C) $RTT_1 + \dots + RTT_n + 12RTT_o$
 D) $RTT_1 + \dots + RTT_n + 14RTT_o$

Problem 4. (Looking up an IP address using nslookup) You can type nslookup followed by the hostname you want to look up (e.g., type "nslookup cityu.edu.hk" in the command prompt), and nslookup will issue a DNS query to find out the IP address.

- What is the IP address of the hostname?
- Can you type the IP address in your browser's address bar and get to the webpage?

HTTP request message

- ❖ two types of HTTP messages: *request*, *response*

- ❖ **HTTP request message:**

- ASCII (human-readable format)

request line
(GET, POST, HEAD commands)

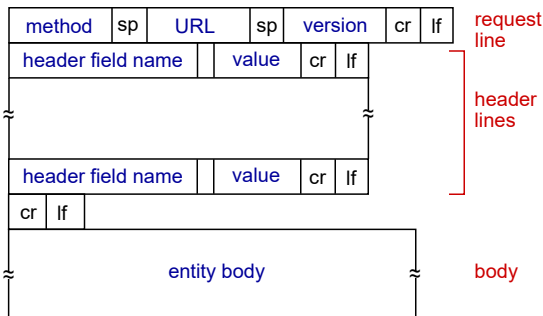
header lines

carriage return, line feed at start of line indicates end of header lines

carriage return character
line-feed character

```
GET /index.html HTTP/1.1\r\n
Host: www-net.cs.umass.edu\r\n
User-Agent: Firefox/3.6.10\r\n
Accept: text/html,application/xhtml+xml\r\n
Accept-Language: en-us,en;q=0.5\r\n
Accept-Encoding: gzip,deflate\r\n
Accept-Charset: ISO-8859-1,utf-8;q=0.7\r\n
Keep-Alive: 115\r\n
Connection: keep-alive\r\n
\r\n
```

HTTP request message: general format



Application Layer 2-2

Q1

- a) The document request was `http://gaia.cs.umass.edu/cs453/index.html`.

The Host : field indicates the server's name and `/cs453/index.html` indicates the file name.

- b) The browser is running HTTP version 1.1, as indicated just before the first `<cr><lf>` pair.
- c) The browser is requesting a persistent connection, as indicated by the `Connection: keep-alive`.

Q1

- d) This is a trick question. This information is not contained in an HTTP message anywhere. So there is no way to tell this from looking at the exchange of HTTP messages alone. One would need information from the IP datagrams (that carried the TCP segment that carried the HTTP GET request) to answer this question.
- e) Mozilla/5.0. The browser type information is needed by the server to send different versions of the same object to different types of browsers.

DNS: services, structure

DNS services

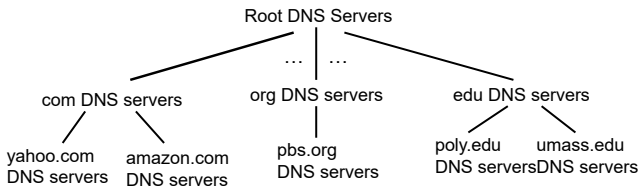
- ❖ hostname to IP address translation
- ❖ host aliasing
 - canonical, alias names
- ❖ mail server aliasing
- ❖ load distribution
 - replicated Web servers: many IP addresses correspond to one name

why not centralize DNS?

- ❖ single point of failure
- ❖ traffic volume
- ❖ distant centralized database
- ❖ maintenance

A: *doesn't scale!*

DNS: a distributed, hierarchical database



client wants IP for www.amazon.com; 1st approx:

- ❖ client queries root server to find com DNS server
- ❖ client queries .com DNS server to get amazon.com DNS server
- ❖ client queries amazon.com DNS server to get IP address for www.amazon.com

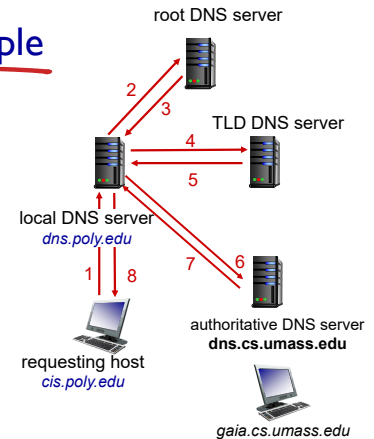
Application Layer 2-6

DNS name resolution example

- ❖ host at cis.poly.edu wants IP address for gaia.cs.umass.edu

iterated query:

- ❖ contacted server replies with name of server to contact
- ❖ "I don't know this name, but ask this server"



Application Layer 2-7

Q2

- As no DNS records are cached at any of the servers, all interactions are needed.
- 1) The total time to get the IP address is $RTT_L + 3RTT_r$.
- 2) Just RTT_L .

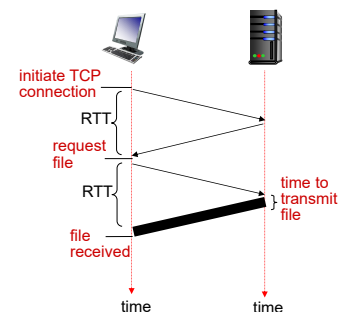
Note: A TLD DNS server is a Top-Level Domain DNS server, which handles queries for a specific top-level domain (TLD) such as .edu, .com, or .org.

Non-persistent HTTP: response time

RTT (definition): time for a small packet to travel from client to server and back

HTTP response time:

- ❖ one RTT to initiate TCP connection
- ❖ one RTT for HTTP request and first few bytes of HTTP response to return
- ❖ file transmission time
- ❖ non-persistent HTTP response time = $2RTT + \text{file transmission time}$



Application Layer 2-9

Persistent HTTP

non-persistent HTTP issues:

- ❖ requires 2 RTTs per object
- ❖ OS overhead for *each* TCP connection
- ❖ browsers often open parallel TCP connections to fetch referenced objects

persistent HTTP:

- ❖ server leaves connection open after sending response
- ❖ subsequent HTTP messages between same client/server sent over open connection
- ❖ client sends requests as soon as it encounters a referenced object
- ❖ as little as one RTT for all the referenced objects

Application Layer 2-10

Q3a

- Time needed

= Time for DNS + (TCP connection setup time + request-response RTT for base HTML file) + (TCP connection setup time + request-response RTT for an object) * 8

$$= (RTT_1 + \dots + RTT_n) + 2RTT_0 + (2RTT_0) * 8$$

$$= (RTT_1 + \dots + RTT_n) + 18RTT_0$$

Q3b

- Time needed

= Time for DNS + (TCP connection setup time + request-response RTT for base HTML file) + (TCP connection setup time + request-response RTT for an object) * 2

$$= (RTT_1 + \dots + RTT_n) + 2RTT_0 + (2RTT_0) * 2$$

$$= (RTT_1 + \dots + RTT_n) + 6RTT_0$$

Q3c

- In persistent HTTP, after the TCP connection has been setup, the nine objects (i.e., one base HTML file plus eight very small objects) have to be requested one after another. Thus,

- Time needed

= Time for DNS + TCP connection setup time + request-response RTT * 9

$$= (RTT_1 + \dots + RTT_n) + RTT_0 + RTT_0 * 9$$

$$= (RTT_1 + \dots + RTT_n) + 10RTT_0$$